

Arquitectura del Comparador de Calidad de Gas

Comparador Regulatorio de Calidad de Gas Natural - Enagás Diseño modular con capa de proveedor de LLM desacoplada

1. Principio de diseño (la regla de oro)

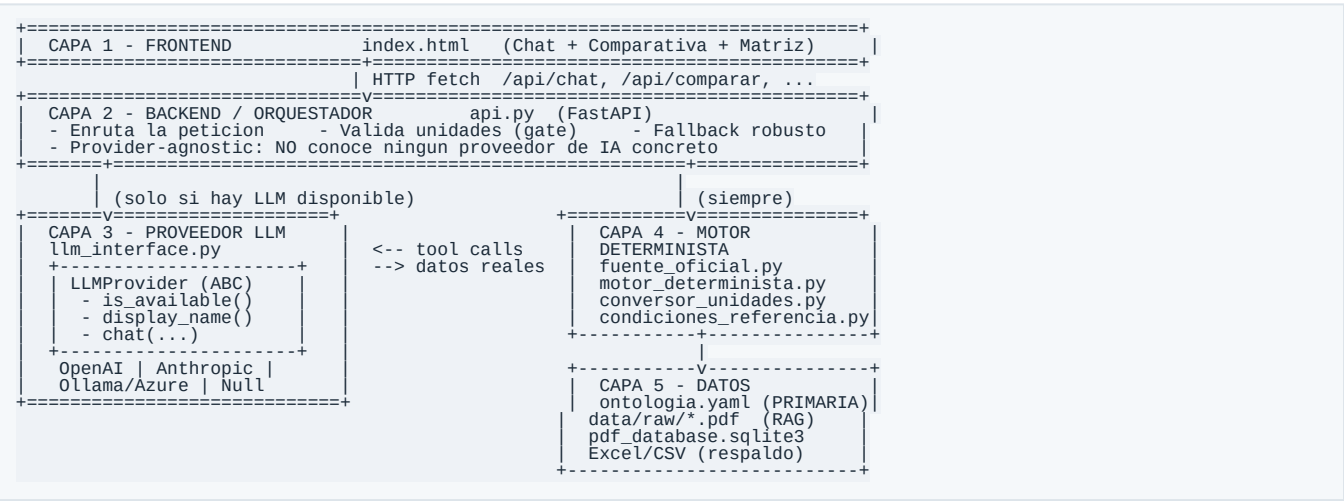
La IA solo entiende y redacta. NUNCA inventa números. Todos los valores salen del motor determinista (ontología verificada + PDFs oficiales).

Sistema híbrido con separación absoluta entre tres responsabilidades:

Responsabilidad	Quién	Puede inventar números
Conversar (interpretar y redactar)	LLM (vía capa de abstracción)	■ Nunca
Datos y cálculos regulatorios	Motor determinista + ontología	— (son la verdad)
Elegir/conectar el proveedor de IA	llm_interface.py	— (solo fontanería)

Esto elimina las alucinaciones numéricas y hace las respuestas auditables.

2. Vista modular por capas



Clave del rediseño: el LLM ya no está "pegado" al backend. La CAPA 3 (llm_interface.py) es la única que conoce a OpenAI/Anthropic/etc. El backend solo habla con la interfaz genérica LLMProvider.

3. Capa por capa (resumen)

Capa 1 - Frontend — [index.html](#)

Página única con dos vistas: **Chat** (lenguaje natural) y **Comparativa** (desplegables -> tabla + matriz heatmap). Sin lógica de negocio: solo fetch y pintado. Se sirve sin caché para que todos vean siempre la última versión.

Capa 2 - Backend / Orquestador — [api.py](#)

Coordina todo y es **provider-agnostic**. Endpoints:

Endpoint	Función
----------	---------

POST /api/chat	Conversación (IA si hay proveedor; si no, fallback determinista)
POST /api/comparar	Tabla comparativa estructurada
GET /api/matriz	Matriz países x parámetros (heatmap)
GET /api/parametros	Poblar desplegables
GET /api/status	Modo activo y nombre del proveedor LLM

Capa 3 - Proveedor LLM — `llm_interface.py` (NUEVO)

Abstrae todo lo específico del proveedor. Ver §5 y §6.

Capa 4 - Motor determinista (la "verdad")

- `fuentes_oficiales.py` — lee la ontología y devuelve el dato oficial con su cita.
- `motor_determinista.py` — Excel/CSV de respaldo + evaluación de cumplimiento.
- `conversor_unidades.py` — conversiones exactas (MJ/m³ <-> kWh/m³, ppm <-> mg/Nm³...).
- `condiciones_referencia.py` — normalización ISO 13443 (Tabla A.1) entre países.

Capa 5 - Datos

Ontología YAML (primaria), PDFs oficiales, índice SQLite del RAG y Excel/CSV de respaldo.

4. Funcionalidades que dependen del LLM

Esta es la lista **completa** de todo lo que hoy usa un LLM en la aplicación. Todo pasa por la capa de abstracción (`llm_interface.py`); ninguna otra parte llama directamente a un proveedor.

4.1 Chat conversacional con function-calling

- **Propósito:** interpretar la pregunta del usuario en lenguaje natural, decidir qué herramientas deterministas llamar y **redactar** la respuesta final (tabla Markdown con veredicto, evidencias, notas y conclusión).
- **Dónde se usa:** endpoint POST /api/chat -> `responder_con_llm()` -> `provider.chat(...)`.
- **Capacidad de API:** *chat completions con tool-calling* (function calling). Iterativo, hasta 5 vueltas.
- **Entradas:** `system_prompt` (instrucciones fijas, sin números), `history` (turnos previos de la sesión), `user_message`, `tools` (6 esquemas de función), `tool_functions` (mapa nombre->función Python), `temperature=0`.
- **Salidas:** un `str` en Markdown (la respuesta redactada). Los números y citas provienen SIEMPRE de las herramientas, no del modelo.

4.2 Selección de herramienta según intención

- **Propósito:** distinguir consulta de límite (`consultar_excel`) de evaluación de cumplimiento (`evaluar_cumplimiento`), pedir conversiones, o buscar en PDF.
- **Dónde se usa:** dentro del mismo bucle de `chat` (lo decide el modelo).
- **Capacidad de API:** tool-calling (el modelo emite `tool_calls`).
- **Entradas:** los 6 esquemas de `LLM_TOOLS`. **Salidas:** invocaciones de función con argumentos JSON que el motor determinista ejecuta.

4.3 Detección de disponibilidad / modo

- **Propósito:** saber si hay LLM operativo para elegir entre modo IA y fallback determinista, y mostrarlo en `/api/status`.
- **Dónde se usa:** `provider.is_available()` y `provider.display_name()` en `backend_mode`, `chat_endpoint` y `status_endpoint`.
- **Capacidad de API:** ninguna llamada de red (solo comprueba credenciales/SDK).
- **Entradas:** variables de entorno. **Salidas:** `bool` y `str`.

Nota: hoy NO se usan *embeddings* (el RAG es por palabras clave sobre SQLite) ni *structured outputs* nativos. Si se añadieran, se ampliaría la interfaz `LLMProvider` con métodos nuevos (p. ej. `embed()`), sin tocar el resto de la app.

5. La capa de abstracción `llm_interface.py` (NUEVO)

Objetivo: **desacoplar la aplicación de OpenAI** para que quien no tenga clave de OpenAI pueda usar otro proveedor (Anthropic, Gemini vía compatible, Azure, Ollama o un modelo local) **modificando solo este fichero** (o ni eso: con variables de entorno).

Interfaz genérica

```
class LLMProvider(ABC):
    def is_available(self) -> bool: ...      # ¿hay LLM listo?
    def display_name(self) -> str: ...      # texto para /api/status
    def chat(system_prompt, history, user_message,
             tools, tool_functions,
             temperature=0.0, max_tool_iterations=5) -> str: ...
```

Proveedores incluidos

Proveedor	Clase	LLM_PROVIDER	Variables de entorno
OpenAI oficial	OpenAIProvider	openai (def.)	API_OPENAI, OPENAI_MODEL
Anthropic Claude	AnthropicProvider	anthropic	ANTHROPIC_API_KEY, ANTHROPIC_MODEL
Ollama / Azure / local	OpenAICompatibleProvider	ollama/azure	LLM_BASE_URL, LLM_API_KEY, LLM_MODEL
Sin LLM (forzar determinista)	NullProvider	null	—

Los endpoints "estilo OpenAI" (Ollama, Azure, LM Studio, vLLM, Together, Groq...) comparten el mismo bucle de tool-calling vía `_OpenAIStyleProvider`.

Cómo cambiar de proveedor (sin tocar código)

```
# OpenAI (por defecto)
API_OPENAI=sk-... OPENAI_MODEL=gpt-4o-mini

# Anthropic Claude
LLM_PROVIDER=anthropic ANTHROPIC_API_KEY=... ANTHROPIC_MODEL=claude-opus-4-8

# Ollama local (sin coste, sin clave real)
LLM_PROVIDER=ollama LLM_BASE_URL=http://localhost:11434/v1 LLM_MODEL=llama3.1
```

Cómo añadir un proveedor nuevo (3 pasos)

1. Crea una subclase de `LLMProvider` (copia `OpenAIProvider`); implementa `is_available`, `display_name` y `chat` (con su bucle de tool-calling y la traducción de esquemas de herramienta a su formato).
2. Regístrala en el diccionario `_PROVIDERS`.
3. Documenta sus variables de entorno en `get_provider`.

6. Puntos de conexión (call sites)

Todo el acoplamiento al LLM está en `llm_interface.py`. En el resto del código solo aparece la interfaz genérica:

Lugar en <code>api.py</code>	Llamada a la abstracción
Init del módulo	<code>provider = get_provider()</code>
<code>backend_mode</code> / <code>status_endpoint</code>	<code>provider.is_available(), provider.display_name()</code>
<code>chat_endpoint</code>	decide IA vs. fallback con <code>provider.is_available()</code>

<code>responder_con_llm()</code>	<code>provider.chat(system_prompt, history, msg, LLM_TOOLS, TOOL_FUNCS)</code>
----------------------------------	--

Ningún otro fichero importa `openai`, `anthropic`, etc. **Cambiar de proveedor no requiere tocar `api.py`.**

7. Recorrido de una pregunta

Ejemplo: "¿Cumple 14 kWh/m³ de PCS en Francia?"

1. Frontend -> `POST /api/chat`.
2. Backend: **gate de validación de unidades** (si falta unidad, la pide).
3. Backend: ¿`provider.is_available()`? Si no -> fallback determinista.
4. `responder_con_llm` -> `provider.chat(...)` (capa 3).
5. El LLM detecta valor medido -> pide `evaluar_cumplimiento`.
6. La capa 3 ejecuta la función -> `fuentes_oficiales.py` lee la **ontología** -> límite francés + cita; si la unidad difiere, `convertor_unidades` normaliza.
7. El resultado real vuelve al LLM.
8. El LLM **redacta** la tabla con veredicto, conversión, **evidencia citada** y **notas** del reglamento.
9. Frontend la pinta.

En ningún momento la IA inventa el límite: lo obtiene de la ontología.

8. RAG (búsqueda documental) — `agente_pdf.py`

Retrieval-Augmented Generation: buscar el fragmento exacto de un PDF y dárselo al LLM como prueba. - **Indexar (una vez):** `pdfplumber` extrae el texto de `data/raw/`, se parte en *chunks* (~1800 caracteres con solape) y se guarda en **SQLite**. - **Buscar (por consulta):** la herramienta `buscar_pdfs` recupera los chunks. - **Nota:** búsqueda **por palabras clave** (LIKE), no por embeddings vectoriales.

9. Robustez: modo IA y modo determinista

- **Con proveedor disponible:** responde el LLM (`modo=ia`).
- **Sin proveedor o si el LLM falla:** cae al **fallback determinista** por palabras clave, que arma la tabla directamente con el motor. El chat **nunca se rompe**. Ambas rutas muestran evidencia citada y notas del reglamento.

10. Mapa de ficheros

Fichero	Rol
<code>index.html</code>	Frontend (chat + comparativa + matriz)
<code>api.py</code>	Backend FastAPI, orquestador, endpoints — provider-agnostic
<code>llm_interface.py</code>	Capa de abstracción del LLM (única que conoce el proveedor)
<code>src/llm/prompts.py</code>	SYSTEM_PROMPT (sin números)
<code>fuentes_oficiales.py</code>	Lee la ontología (fuente primaria)
<code>motor_determinista.py</code>	Excel/CSV de respaldo + evaluación
<code>convertor_unidades.py</code>	Conversiones exactas de unidades
<code>condiciones_referencia.py</code>	Normalización ISO 13443 entre países

<code>agente_pdf.py</code>	Indexado y búsqueda de PDF (RAG)
<code>data/ontologia/ontologia_enagas.yaml</code>	La fuente de verdad (10 params x 21 juris)
<code>data/raw/*.pdf</code>	Documentos normativos oficiales
<code>data/pdf_database.sqlite3</code>	Índice RAG (chunks de PDF)

11. Filosofía en una frase

La IA pone las palabras; los datos verificados ponen los números; y el proveedor de IA es intercambiable. Trazable, auditable, sin alucinaciones y sin quedar atado a un único proveedor.